

Retrieval-Augmented Generation (RAG) System for Question Answering

Project Proposal

Team 10

Submitted To:

Dr. Prashanth Cheluvasai Ranga

Supervisor

COMP 8967

Summer 2026 Term

(June 1 -- July 31, 2026)

Academic Affiliate:

Shaon Shuvo

Submitted By:

Anurag Sharma -- 110171321

Prabhjot Singh -- 110195228

Harpreet Singh -- 110177411

Chetan Thakur -- 110213868

University of Windsor

June 2026

Table of Contents

1. Introduction	3
2. Motivation	4
3. Background Study	5
4. Project Goals	7
5. Project Roadmap	8
6. Resources	11
7. References	13

1. Introduction

Question Answering (QA) systems have become an integral part of how we interact with information in the digital age. From search engines to virtual assistants, users expect accurate, contextually relevant, and timely responses to their queries. Traditional QA systems, however, face significant limitations: they rely on static knowledge bases, struggle with domain-specific questions, and are prone to generating incorrect or hallucinated information when faced with unfamiliar queries.

Our project addresses these challenges by implementing a Retrieval-Augmented Generation (RAG) system. RAG represents a paradigm shift in how we approach question answering by combining the strengths of information retrieval with the generative capabilities of modern language models. Unlike traditional approaches that depend solely on parametric knowledge stored within a model, RAG systems dynamically retrieve relevant documents from an external knowledge base and use them to generate more accurate, grounded, and verifiable responses.

This project aims to build a complete, end-to-end RAG system capable of handling real-world question answering tasks using the MS MARCO dataset, a widely recognized benchmark for passage retrieval and QA. By integrating state-of-the-art embedding models, efficient vector indexing using FAISS, and carefully selected generation strategies, we seek to demonstrate how RAG can improve both the accuracy and reliability of QA systems.

2. Motivation

2.1 Why is this Problem Interesting?

The challenge of building effective QA systems sits at the intersection of several active research areas in artificial intelligence, including natural language processing, information retrieval, and representation learning. What makes this problem particularly interesting is the tension between two competing requirements: the need for systems that understand and reason about language, and the need for systems that can access and process vast amounts of factual information.

Pure generative models, while impressive in their ability to produce fluent text, suffer from fundamental limitations. They are trained on fixed corpora and cannot access information beyond what was present during training. More critically, they are prone to hallucination-generating plausible-sounding but factually incorrect information. This is particularly problematic in domains where accuracy is paramount, such as healthcare, legal advice, or educational applications.

2.2 Why is it Challenging?

Building an effective RAG system presents several technical challenges:

1. **Retrieval Accuracy:** The system must identify the most relevant passages from potentially millions of documents. Poor retrieval leads to poor generation, even with sophisticated language models.
2. **Computational Efficiency:** Real-time querying requires fast similarity search over high-dimensional vector representations. Balancing accuracy with speed is a critical design consideration.

3. Integration: Effectively combining retrieved context with generative models requires careful prompt engineering and model selection.
4. Evaluation: Measuring the quality of generated answers is inherently more complex than evaluating retrieval alone, requiring both automated metrics and human judgment.

2.3 Who Benefits?

Several stakeholders stand to benefit from improved RAG systems:

End Users: Anyone seeking information through search interfaces benefits from more accurate, comprehensive answers with source attribution.

Organizations: Companies handling customer support, knowledge management, or internal documentation can deploy RAG systems to improve information access.

Researchers: The RAG framework provides a testbed for exploring novel approaches to retrieval, representation learning, and generation.

Educational Platforms: Students and educators can leverage RAG systems as tutoring aids that provide explanatory answers with citations.

3. Background Study

3.1 The RAG Framework

Retrieval-Augmented Generation was formally introduced by Lewis et al. (2020) as a way to combine parametric and non-parametric memory in language models. The key insight is that rather than storing all knowledge in the model's parameters (which is expensive and static), we can maintain a separate, updatable knowledge base and retrieve relevant information at generation time.

The original RAG architecture consists of two main components:

1. Retriever: A dense passage retrieval model that learns to encode documents and queries into a shared embedding space, allowing efficient similarity-based retrieval.
2. Generator: A sequence-to-sequence model (typically based on the Transformer architecture) that conditions its output on both the input query and the retrieved documents.

3.2 Embedding Models

Converting text into meaningful vector representations is fundamental to RAG systems. Two prominent approaches we will evaluate are:

BGE (BAAI General Embedding): A family of embedding models developed by the Beijing Academy of Artificial Intelligence. The BGE-small variant offers a favorable trade-off between model size and performance, making it suitable for resource-constrained deployments.

E5 (Embeddings from Bidirectional Encoder Representations): A model family that emphasizes training with weak supervision signals, allowing it to learn robust representations across diverse domains. E5-base provides a middle ground in terms of computational cost and embedding quality.

Both models leverage the SentenceTransformers framework, which provides a convenient interface for encoding text and can be fine-tuned for specific tasks.

3.3 Vector Similarity Search with FAISS

FAISS (Facebook AI Similarity Search) is a library developed by Meta Research for efficient similarity search and clustering of dense vectors. In our RAG system, FAISS will serve as the vector database, indexing embeddings of all passages in our corpus.

Key FAISS indexing strategies we will explore:

Flat Index: Provides exact nearest neighbor search by computing distances to all vectors. Suitable for smaller datasets.

IVF (Inverted File Index): Partitions the vector space and only searches relevant partitions, trading slight accuracy loss for significant speed improvements.

HNSW (Hierarchical Navigable Small World): Builds a graph structure enabling very fast approximate search with minimal accuracy degradation.

3.4 The MS MARCO Dataset

The Microsoft Machine Reading Comprehension (MS MARCO) dataset is a large-scale real-world QA collection. Unlike synthetic datasets, MS MARCO contains real user queries from the Bing search engine, along with human-written answers and relevance judgments for passages. This realism makes it an ideal benchmark for evaluating RAG systems, as it reflects the variability and ambiguity of actual user queries.

3.5 Key References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
2. Nguyen, T., et al. (2016). MS MARCO: A Human Generated Machine Reading Comprehension Dataset. arXiv.
3. Johnson, J., et al. (2019). Scaling Dense Vector Retrieval to Billions of Items. arXiv.
4. Xiao, S., et al. (2023). BGE-M3: Multi-Functionality, Multi-Linguality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. EMNLP.
5. Wang, L., et al. (2024). Text Embeddings by Weakly-Supervised Contrastive Pre-training. arXiv.

4. Project Goals

4.1 Overall Vision

Our project aims to develop and evaluate a complete Retrieval-Augmented Generation system for open-domain question answering. The system should demonstrate improved answer quality compared to baseline approaches while maintaining acceptable computational efficiency for real-time or near-real-time querying.

4.2 Specific Objectives

1. Build a Functional RAG Pipeline: Implement all components necessary for end-to-end question answering, including document indexing, query processing, retrieval, and answer generation.
2. Evaluate Multiple Configurations: Compare different embedding models (BGE-small vs. E5-base) and indexing strategies (Flat vs. IVF vs. HNSW) to understand their trade-offs in terms of accuracy, latency, and resource usage.
3. Benchmark on MS MARCO: Achieve competitive performance on standard MS MARCO evaluation metrics (Mean Reciprocal Rank for retrieval, and appropriate generation quality metrics).
4. Document and Replicate: Create comprehensive documentation and reproducible experiments, enabling others to understand and build upon our work.
5. Deliver a Usable Prototype: Provide a demonstration interface showing the system's capabilities on sample queries.

4.3 What We Are NOT Committing To

To ensure project feasibility within the eight-week timeframe, we explicitly exclude the following:

- Training new embedding models from scratch (we will use pre-trained models)
- Achieving state-of-the-art performance on MS MARCO leaderboards (our focus is on understanding, not competition)
- Multi-modal retrieval (handling images, videos, or structured data)
- Real-time deployment at web scale (we target prototype-scale deployment)
- Fine-tuning large language models for generation (we will use pre-trained or API-based models)

5. Project Roadmap

The project spans from June 1, 2026 to July 31, 2026 (8 weeks). Below are the planned weekly deliverables and team responsibilities.

Phase 1: Foundation (Weeks 1-2)

Week / Date	Deliverables	Team Members
Week 1 <i>Jun 1-7</i>	Project proposal submission; Initial literature review; Team workflow setup; Git repository initialization	All

Week 2 <i>Jun 8-14</i>	Environment setup complete; Required packages installed (Python, PyTorch, FAISS, Transformers); Basic project structure created	Harpreet, Chetan
---------------------------	---	------------------

Phase 2: Data and Baselines (Weeks 3-4)

Week / Date	Deliverables	Team Members
Week 3 <i>Jun 15-21</i>	Dataset download and exploration completed; Data format documented; Initial data loading scripts; Data preprocessing pipeline	Anurag, Prabhjot
Week 4 <i>Jun 22-28</i>	Baseline retrieval implementation (TF-IDF or basic embeddings); Initial evaluation framework; Documentation of baseline results	All

Phase 3: Core Implementation (Weeks 5-6)

Week / Date	Deliverables	Team Members
Week 5 <i>Jun 29-Jul 5</i>	Embedding model integration (BGE-small initial testing); Document encoding pipeline; FAISS indexing implementation; Basic similarity search working	Harpreet, Chetan
Week 6 <i>Jul 6-12</i>	E5-base model integration; Comparative analysis of embedding quality; Retrieval evaluation script; Initial MRR scores on validation set	Anurag, Prabhjot

Phase 4: Integration and Evaluation (Weeks 7-8)

Week / Date	Deliverables	Team Members
Week 7 <i>Jul 13-19</i>	Generation component integration; Basic RAG pipeline functional end-to-end; Prompt engineering and optimization; Generation quality evaluation setup	All
Week 8 <i>Jul 20-26</i>	Full MS MARCO validation evaluation; HNSW and IVF indexing experiments; Performance benchmarking (accuracy vs. speed); User interface prototype; Final documentation and code submission	All

Buffer and Finalization (Week 9)

Week / Date	Deliverables	Team Members
Week 9 <i>Jul 27-31</i>	Buffer week for final testing, bug fixes, documentation improvements, and presentation preparation; Final submission deadline	All

5.1 Team Roles and Responsibilities

Team Member	Primary Responsibilities
Anurag Sharma	Literature review, evaluation methodology, experiment design, metrics implementation, and reporting
Prabhjot Singh	Data acquisition, preprocessing, data pipeline implementation, and dataset management
Harpreet Singh	Core implementation (embedding models, FAISS integration, retrieval system), code architecture
Chetan Thakur	Generation component integration, prompt engineering, user interface prototype, and documentation

6. Resources

6.1 Required Skills

Team members will develop or apply expertise in the following areas:

Python Programming: Core implementation language; familiarity with data science libraries (NumPy, Pandas, PyTorch/TensorFlow)

Natural Language Processing: Understanding of text processing, tokenization, and language model usage

Machine Learning Fundamentals: Embeddings, similarity metrics, evaluation metrics

Version Control: Git for collaborative development and code management

Technical Writing: Documenting experiments, writing reports, and preparing presentations

6.2 Technology Stack

Programming and Core Libraries

Python 3.9+, NumPy, Pandas (data manipulation), PyTorch or TensorFlow (deep learning frameworks)

NLP and Embedding Tools

SentenceTransformers (embedding model framework), HuggingFace Transformers (model access), tiktoken or similar (tokenization)

Vector Database

FAISS (Facebook AI Similarity Search)

Dataset

MS MARCO passage collection and query set

Generation

OpenAI API or open-source LLM (e.g., LLaMA, Mistral) via API or local inference, LangChain or similar framework (optional, for RAG orchestration)

Development Tools

Jupyter Notebook / JupyterLab (experimentation), Git and GitHub/GitLab (version control), pytest (testing), Black/Ruff (code formatting and linting)

Documentation and Reporting

Markdown (README and inline documentation), Matplotlib/Seaborn (visualization for reports)

6.3 Hardware Requirements

CPU: Modern multi-core processor for efficient data processing

RAM: Minimum 16GB; 32GB+ recommended for large-scale indexing operations

GPU: CUDA-compatible GPU beneficial for embedding generation; not strictly required if using APIs

Storage: 50GB+ for dataset storage, model checkpoints, and indexes

6.4 Project Management Tools

GitHub Projects: For task tracking and sprint planning

Shared Document Repository: Google Drive or similar for collaborative writing

Communication: Group chat/meeting platform for regular coordination

7. References

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459-9474.
2. Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwari, M., Majumder, R., ... & Goodman, L. (2016). MS MARCO: A Human Generated Machine Reading Comprehension Dataset. *arXiv preprint arXiv:1611.08231*.
3. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *arXiv preprint arXiv:1902.09129*.
4. Xiao, S., Liu, Z., Zhang, P., Pei, Y., & Shi, Y. (2023). BGE-M3: Multi-Functionality, Multi-Linguality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 12098-12119.
5. Wang, L., Yang, N., Wei, F., Chang, B., Yang, M., Li, J., ... & Zhang, Y. (2024). Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03527*.
6. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 3982-3992.
7. Guo, J., Fiscus, J., & Guo, Y. (2023). MS MARCO: A Comprehensive Overview and Analysis of the MS MARCO Dataset. *arXiv preprint arXiv:2310.10983*.
8. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6690-6700.
9. Zhao, W. X., et al. (2023). A Survey of Large Language Models. *arXiv preprint arXiv:2303.18223*.
10. Gao, T., Yao, S., & Chen, D. (2021). SimCSE: Simple Contrastive Learning of Sentence Embeddings. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 6894-6910.